

Sprint 3 — Scenario-specific Test Data: Proof

Branch: `scenario-specific-test-data` · **Defect class:** confident-but-generic test data

Scope: `dataPhraseFor()` + one helper `sampleValueForField()` in `src/engines/scenario-builder.ts`.
No new engine, no planner / App-Profile-schema / KB / LLM changes.

1. The defect (Gate 0)

Every positive / coverage-default form-fill step fell through `dataPhraseFor` to a single generic placeholder. A generated “Add Employee” suite shipped rows a human would instantly recognise as machine-templated and could not execute as written:

```
Enter a valid First Name in the First Name field
Enter a valid Last Name in the Last Name field
Enter a valid Middle Name in the Middle Name field
```

Root cause: `dataPhraseFor` dispatched correctly for security/negative intents (SQL, XSS, duplicate, whitespace) but the positive branch returned `"a valid " + label`. The field carried no constraint metadata, and none was threaded from the requirement.

2. The fix

- **Deterministic sample library** selected by `hash(fieldName + scenarioId)` — repeatable but varied across a form and across scenarios, so 500 generations never read like one hardcoded “Emma Watson”. Distinct pools per concept (first / middle / last) so a row never repeats a value.
- **Field-TYPE-aware values** — driven by the field’s `type` (and name as fallback):
email → address, tel → phone number, number/age → numeric, url, date, etc.
- **Type-aware invalids** for negatives — email → `"not-an-email"`, phone → `"123"`,
number → `"-1"`, url → malformed — instead of `"an invalid Email"`.
- **Boundary values grounded in REAL constraints** — max / min length are parsed from the requirement text and attributed to a field only when its name appears alongside the number. Whichever boundary the scenario requires is generated (MAX / MAX+1 / MIN / MIN-1).
- **Never invent** a length / range / regex / uniqueness rule the requirement did not state.
- **Unknown field fallback** → `"<Label> Example"` (e.g. `"Notes Example"`), never `"valid Notes"`.
- **Preserved unchanged:** SQL, XSS, duplicate, whitespace, unicode and numeric intent payloads.

3. Before → After

Intent (field)	Before	After
Positive (First Name)	a valid First Name	"Emma" / "Aarav" / "Priya" ... (varied)
Positive (Middle Name)	a valid Middle Name	"Rose" / "Lee" ... (distinct pool)
Positive (Email, type=email)	a valid Email	"david.johnson@example.com"
Positive (Phone, type=tel)	a valid Phone	"8005551234"
Positive (Age, type=number)	a valid Age	"28"
Positive (Department)	a valid Department	"Finance" / "Marketing" ...
Positive (unknown "Notes")	a valid Notes	"Notes Example"
Negative (Email)	an invalid Email	an invalid email address (e.g. "not-an-email")
Negative (Phone)	an invalid Phone	an invalid phone number (e.g. "123")
Negative (Age)	an invalid Age	an out-of-range number (e.g. "-1")
Boundary (Username, max 20 stated)	a very long Username	values of 19, 20 and 21 characters (below, at and above the 20-character limit)
Boundary (Email/Phone/Age, no constraint stated)	invented "exceeding 255 characters"	no invented number — descriptive edge only
SQL / XSS / duplicate / whitespace	(unchanged)	byte-for-byte unchanged

4. Success metrics

Metric	Target	Result
"valid First Name" occurrences (Add Employee output)	0	0
"valid data" as test data (excludes scenario titles)	0	0
Positive scenarios using executable values	100%	100%
Type-aware invalids (email/phone/url/number)	present	present
Boundary grounded in real constraint when one exists	100%	100%
Invented constraints when requirement is silent	0	0
SQL / XSS regressions	0	0
Duplicate / whitespace regressions	0	0

5. Tests

- **New:** `tests/unit/scenario-specific-test-data.test.ts` — **16/16 pass**
(placeholder eliminated · realistic positive · type-aware values & invalids · boundary grounded in stated max · no invented constraint · SQL/XSS/duplicate/whitespace preserved · unknown-field fallback).
- **Updated:** one obsolete assertion in `scenario-builder.test.ts` that encoded the old generic behaviour (asserted the literal word "valid" in a positive step). Rewritten to assert the new scenario-specific value. Suite now **28/28 pass**.
- **Regression sweep — all related suites green:**

Suite	Result
scenario-builder	28/28
scenario-specific-test-data (new)	16/16
intent-step-generator	10/10
feature-grounding-engine	12/12
scenario-correctness	25/25
scenario-integrity	32/32
generation-quality-engine	25/25
expected-result-validator	26/26
expected-result-excellence	21/21
requirement-understanding-engine	25/25
validation-planner	26/26
scenario-planner-field-aware	5/5
scenario-planner-standard-coverage	7/7
leave-workflow-depth	2/2
qa-knowledge-stepflow	4/4
qa-architect-scorer	5/5

- `npx tsc --noEmit` → exit 0.
- `scripts/testdata-proof.ts` → **ALL 7 PROOFS PASS.**